



MODUL TEORI

Disusun Oleh:

Aprilia Kurnianti, S.T., M.Eng.

Dosen Pengampuh Matkul IBD

DAFTAR ISI

Materi 8 TRANSACTION MANAGEMENT, LOCKING, DEADLOCK, AND ISOLATION LEVEL	3
A. Pendahuluan.....	3
B. Transaction Management	3
C. ACID	4
D. BEGIN TRANSACTION.....	5
E. COMMIT	5
F. ROLLBACK.....	5
G. SAVE TRANSACTION.....	6
H. Perbedaan COMMIT, ROLLBACK, dan SAVE TRANSACTION.....	6
I. Locking	6
J. Blocking	7
K. Isolation Level.....	7

Materi 8

Transaction Management, Locking, Deadlock, dan Isolation Level

A. Pendahuluan

Dalam sistem basis data, sebuah proses bisnis sering kali tidak hanya membutuhkan satu operasi SQL, tetapi juga terdiri dari beberapa operasi yang saling berkaitan. Sebagai contoh, dalam proses penjualan pada database DB_Petshop, sistem dapat menyimpan data transaksi ke dalam tabel Penjualan, menyimpan barang yang dibeli ke dalam tabel DetailPenjualan, serta mengurangi jumlah stok pada tabel Barang. Ketiga operasi tersebut merupakan bagian dari satu proses bisnis yang sama. Apabila salah satu operasi gagal, sedangkan operasi lainnya tetap tersimpan, maka dapat terjadi ketidakkonsistenan data. Oleh karena itu, diperlukan mekanisme transaction management untuk memastikan sekumpulan operasi database dapat diproses sebagai satu kesatuan.

Transaction atau transaksi adalah sekumpulan operasi database yang diperlakukan sebagai satu unit kerja. Suatu transaksi dapat terdiri dari satu atau beberapa perintah SQL. Transaksi memiliki dua kemungkinan hasil, yaitu seluruh perubahan berhasil disimpan atau perubahan yang dilakukan selama transaksi dibatalkan. Dalam SQL Server, transaksi dapat dikendalikan menggunakan beberapa perintah utama, yaitu BEGIN TRANSACTION, COMMIT TRANSACTION, ROLLBACK TRANSACTION, dan SAVE TRANSACTION. Penggunaan transaksi sangat penting terutama pada sistem yang melakukan perubahan data secara bersamaan dan melibatkan beberapa tabel yang saling berhubungan.

B. Transaction Management

Transaction Management merupakan mekanisme untuk mengatur proses transaksi, mulai dari transaksi dimulai, proses operasi database dilakukan, hingga transaksi tersebut diselesaikan. Transaksi biasanya dimulai dengan BEGIN TRANSACTION. Setelah seluruh operasi berhasil dan data telah memenuhi aturan yang ditentukan, transaksi dapat diselesaikan menggunakan COMMIT TRANSACTION. Sebaliknya, apabila terjadi kesalahan atau kondisi tertentu yang menyebabkan transaksi tidak dapat dilanjutkan, transaksi dapat dibatalkan menggunakan ROLLBACK TRANSACTION.

Sebagai contoh, ketika sistem melakukan pembelian barang, proses dapat dimulai dengan BEGIN TRANSACTION, kemudian melakukan pengurangan stok barang. Jika proses tersebut berhasil, transaksi dapat diakhiri dengan perintah COMMIT. Namun, apabila terjadi kesalahan pada proses berikutnya, transaksi dapat dibatalkan menggunakan ROLLBACK. Dengan mekanisme tersebut, perubahan data yang tidak lengkap dapat dihindari.

C. ACID

Transaction Management memiliki 4 karakteristik utama yang dikenal dengan istilah ACID, yaitu Atomicity, Consistency, Isolation, dan Durability. Keempat karakteristik tersebut merupakan prinsip penting dalam menjaga keandalan transaksi pada sistem basis data.

Atomicity berarti transaksi diperlakukan sebagai satu kesatuan. Seluruh operasi dalam transaksi harus berhasil atau seluruh perubahan transaksi harus dibatalkan. Sebagai contoh, pada transaksi penjualan terdapat proses penyimpanan data penjualan, penyimpanan detail penjualan, serta pengurangan stok. Apabila pengurangan stok gagal, transaksi dapat dibatalkan sehingga perubahan yang telah dilakukan sebelumnya tidak meninggalkan data transaksi yang tidak lengkap. Dengan demikian, Atomicity dapat dipahami dengan prinsip all-or-nothing, yaitu semuanya berhasil atau semuanya dibatalkan.

Consistency berarti setiap transaksi harus menjaga database tetap berada dalam kondisi yang sesuai dengan aturan dan constraint yang telah ditentukan. Database harus tetap memenuhi aturan seperti Primary Key, Foreign Key, CHECK, UNIQUE, NOT NULL, serta aturan bisnis lainnya. Sebagai contoh, apabila sistem menetapkan bahwa stok barang tidak boleh bernilai negatif, maka transaksi tidak boleh menghasilkan kondisi stok yang melanggar aturan tersebut.

Isolation berkaitan dengan bagaimana suatu transaksi berinteraksi dengan transaksi lain yang berjalan secara bersamaan. Ketika beberapa transaksi mengakses atau mengubah data yang sama, database harus mengatur agar transaksi tersebut tidak menghasilkan kondisi data yang tidak semestinya. Pengaturan isolasi transaksi berkaitan erat dengan konsep locking, blocking, dan tingkat isolasi (isolation level).

Durability berarti perubahan yang telah berhasil dilakukan dan diselesaikan menggunakan COMMIT harus tetap tersimpan. Setelah transaksi berhasil dikomit, perubahan tersebut dianggap sebagai bagian dari keadaan database yang baru. Dengan demikian, hasil transaksi tidak seharusnya hilang hanya karena transaksi tersebut telah selesai.

D. BEGIN TRANSACTION

BEGIN TRANSACTION digunakan untuk memulai transaksi secara eksplisit. Setelah perintah tersebut dijalankan, perintah-perintah SQL berikutnya dapat menjadi bagian dari transaksi sampai transaksi tersebut diselesaikan dengan COMMIT atau ROLLBACK. Contoh sederhana adalah ketika sistem ingin mengurangi stok barang sebanyak 2 unit. Transaksi dapat dimulai dengan BEGIN TRANSACTION, kemudian dilakukan perintah UPDATE terhadap stok barang. Setelah seluruh proses dinyatakan berhasil, transaksi dapat dilanjutkan dengan perintah COMMIT.

Penggunaan BEGIN TRANSACTION memungkinkan beberapa operasi database ditempatkan dalam satu unit kerja. Hal ini sangat berguna ketika beberapa perubahan harus berhasil secara bersamaan untuk menjaga konsistensi data.

E. COMMIT

COMMIT TRANSACTION digunakan untuk menyelesaikan transaksi dengan menyimpan perubahan yang telah dilakukan. Ketika transaksi berhasil dan tidak ditemukan kondisi yang menyebabkan transaksi harus dibatalkan, COMMIT digunakan untuk menandai bahwa perubahan tersebut telah diterima oleh basis data. Sebagai contoh, setelah proses pengurangan stok berhasil dilakukan dan seluruh proses transaksi telah berjalan dengan benar, sistem dapat menjalankan COMMIT TRANSACTION.

Secara sederhana, COMMIT dapat dipahami sebagai keputusan bahwa transaksi berhasil dan perubahan tetap dipertahankan. Setelah transaksi selesai dengan COMMIT, transaksi tersebut tidak dapat dikembalikan menggunakan ROLLBACK biasa karena transaksi sudah selesai.

F. ROLLBACK

ROLLBACK TRANSACTION digunakan untuk membatalkan perubahan yang dilakukan selama transaksi. Perintah ini biasanya digunakan ketika terjadi kesalahan atau kondisi yang menyebabkan transaksi tidak dapat dilanjutkan. Sebagai contoh, apabila sistem telah melakukan perubahan stok tetapi kemudian proses penyimpanan detail transaksi gagal, sistem dapat melakukan ROLLBACK sehingga perubahan yang dilakukan selama transaksi tersebut dibatalkan.

ROLLBACK sangat penting dalam penerapan prinsip Atomicity. Dengan menggunakan ROLLBACK, sistem dapat menghindari kondisi ketika sebagian operasi berhasil tersimpan, sedangkan operasi lainnya gagal. Oleh karena itu, penggunaan COMMIT dan ROLLBACK harus dirancang berdasarkan kondisi keberhasilan atau kegagalan proses transaksi.

G. SAVE TRANSACTION

Selain COMMIT dan ROLLBACK, SQL Server menyediakan SAVE TRANSACTION untuk membuat savepoint pada transaksi. Savepoint merupakan titik tertentu yang dapat digunakan sebagai lokasi untuk melakukan rollback sebagian dari transaksi. Dengan adanya savepoint, transaksi tidak harus selalu dibatalkan sepenuhnya.

Sebagai contoh, sebuah transaksi melakukan operasi pertama, kemudian membuat SAVE TRANSACTION dengan nama tertentu. Setelah itu, transaksi melakukan operasi kedua. Apabila operasi kedua mengalami masalah, sistem dapat melakukan ROLLBACK ke savepoint tersebut. Dengan demikian, perubahan setelah savepoint dapat dibatalkan, sementara perubahan sebelum savepoint tetap berada dalam transaksi. Transaksi kemudian masih dapat dilanjutkan dan pada akhirnya dapat diselesaikan dengan perintah COMMIT.

H. Perbedaan COMMIT, ROLLBACK, dan SAVE TRANSACTION

COMMIT, ROLLBACK, dan SAVE TRANSACTION memiliki fungsi yang berbeda. COMMIT digunakan untuk menyelesaikan transaksi dan mempertahankan perubahan. ROLLBACK digunakan untuk membatalkan perubahan transaksi. Sementara itu, SAVE TRANSACTION digunakan untuk membuat titik penyimpanan sementara di dalam transaksi sehingga rollback dapat dilakukan hingga titik tertentu. Dengan demikian, SAVE TRANSACTION memberikan kontrol yang lebih fleksibel terhadap transaksi yang memiliki beberapa tahap pekerjaan.

I. Locking

Ketika beberapa transaksi berjalan secara bersamaan, kemungkinan terjadi akses ke data atau resource yang sama. SQL Server menggunakan mekanisme locking untuk mengatur akses tersebut. Lock dapat dipahami sebagai mekanisme pengamanan terhadap sumber daya database selama transaksi berlangsung. Dengan adanya locking, SQL Server dapat mengatur transaksi mana yang dapat mengakses atau mengubah resource tertentu pada waktu tertentu.

Sebagai contoh, Session 1 menjalankan transaksi dan mengubah data pada PRD001. Selama transaksi tersebut masih berlangsung, resource yang berkaitan dengan perubahan tersebut dapat terkunci. Apabila Session 2 mencoba melakukan perubahan terhadap resource yang sama, Session 2 dapat mengalami penundaan hingga Session 1 menyelesaikan transaksinya. Kondisi ini merupakan salah satu contoh bagaimana locking bekerja dalam lingkungan database yang memiliki transaksi secara bersamaan.

Locking diperlukan untuk membantu menjaga konsistensi data saat banyak transaksi berjalan secara bersamaan. Tanpa mekanisme pengendalian akses, transaksi yang berjalan secara bersamaan dapat saling memengaruhi dan menghasilkan kondisi data yang tidak sesuai dengan proses bisnis.

J. Blocking

Blocking adalah kondisi ketika suatu sesi harus menunggu karena resource yang dibutuhkan sedang digunakan atau ditahan oleh sesi lain. Blocking sering terjadi ketika sebuah transaksi masih berjalan dan memegang lock terhadap resource yang kemudian dibutuhkan oleh transaksi lain.

Sebagai contoh, Session 1 melakukan BEGIN TRANSACTION, kemudian melakukan UPDATE terhadap PRD001, tetapi belum melakukan COMMIT atau ROLLBACK. Kemudian Session 2 mencoba melakukan UPDATE terhadap PRD001. Karena resource tersebut sedang digunakan oleh Session 1, Session 2 dapat menunggu hingga Session 1 menyelesaikan transaksi. Setelah Session 1 melakukan COMMIT atau ROLLBACK, resource yang ditahan dapat dilepaskan sehingga Session 2 dapat melanjutkan prosesnya.

Blocking bukan selalu merupakan kesalahan. Blocking dapat menjadi bagian yang normal dari mekanisme kontrol konkurensi. Namun, blocking yang berlangsung terlalu lama dapat menurunkan kinerja sistem. Salah satu penyebabnya adalah transaksi yang terlalu lama dibiarkan terbuka.

K. Isolation Level

Isolation Level menentukan bagaimana suatu transaksi diisolasi dari transaksi lain yang berjalan secara bersamaan. Isolation level berpengaruh terhadap cara transaksi membaca dan berinteraksi dengan perubahan yang dilakukan oleh transaksi lainnya. Pemilihan isolation level merupakan bagian penting dalam pengelolaan keseimbangan antara konsistensi data dan concurrency.

SQL Server menyediakan beberapa tingkat isolasi, yaitu READ UNCOMMITTED, READ COMMITTED, REPEATABLE READ, SNAPSHOT, dan SERIALIZABLE. Masing-masing memiliki tingkat isolasi dan karakteristik yang berbeda.

READ UNCOMMITTED merupakan isolation level dengan tingkat isolasi paling rendah. Pada tingkat ini, transaksi dapat membaca data yang belum dikommit oleh transaksi lain. Kondisi tersebut dapat menghasilkan dirty read, yaitu ketika transaksi membaca data yang pada akhirnya dapat dibatalkan oleh transaksi lain.

READ COMMITTED memiliki tingkat isolasi yang lebih tinggi. Secara umum, transaksi tidak membaca perubahan yang belum dikommit oleh transaksi lain. READ COMMITTED merupakan isolation level default SQL Server pada banyak konfigurasi, meskipun perilaku detailnya dapat dipengaruhi oleh konfigurasi seperti READ_COMMITTED_SNAPSHOT.

REPEATABLE READ memberikan isolasi yang lebih kuat dibandingkan dengan READ COMMITTED. Data yang telah dibaca oleh suatu transaksi dijaga agar tidak berubah oleh transaksi lain selama transaksi pertama masih berlangsung. Karena mekanisme isolasinya lebih ketat, penggunaan REPEATABLE READ dapat menyebabkan sumber daya tertentu ditahan lebih lama.

SNAPSHOT menggunakan mekanisme row versioning sehingga transaksi dapat membaca versi data yang konsisten tanpa harus selalu menunggu perubahan yang sedang dilakukan oleh transaksi lain seperti pada pendekatan locking tradisional. Penggunaan SNAPSHOT memerlukan konfigurasi database yang sesuai.

SERIALIZABLE merupakan salah satu isolation level dengan tingkat isolasi yang paling ketat. Tujuannya adalah memberikan hasil yang seolah-olah transaksi berjalan secara berurutan. Tingkat isolasi yang tinggi dapat meningkatkan konsistensi, tetapi pada kondisi tertentu juga dapat meningkatkan blocking dan mengurangi concurrency.

L. Hubungan Isolation Level dan Concurrency

Isolation level berkaitan erat dengan keseimbangan antara konsistensi dan concurrency. Isolation level yang lebih rendah umumnya memberikan lebih banyak kesempatan bagi transaksi untuk berjalan secara bersamaan, tetapi memiliki risiko membaca data yang belum final atau mengalami fenomena concurrency tertentu. Sebaliknya, isolation level yang lebih tinggi memberikan pengendalian yang lebih ketat terhadap transaksi, tetapi dapat menyebabkan transaksi menunggu lebih sering.

Oleh karena itu, isolation level tidak selalu harus menggunakan tingkat yang paling tinggi. Pemilihannya harus disesuaikan dengan kebutuhan aplikasi, karakteristik transaksi, tingkat konsistensi yang dibutuhkan, serta kebutuhan concurrency pada sistem.

M. Deadlock

Deadlock adalah kondisi ketika dua atau lebih transaksi saling menunggu resource yang sedang digunakan oleh transaksi lainnya sehingga transaksi tersebut tidak dapat melanjutkan prosesnya. Deadlock berbeda dengan blocking biasa. Pada blocking, satu transaksi dapat menunggu transaksi lain yang pada akhirnya menyelesaikan pekerjaannya. Pada deadlock, terjadi pola saling menunggu sehingga transaksi tidak dapat menyelesaikan proses tanpa intervensi.

Contoh sederhana adalah Session 1 mengunci PRD001 kemudian mencoba mengakses PRD002, sementara Session 2 telah mengunci PRD002 dan mencoba mengakses PRD001. Session 1 menunggu Session 2, sedangkan Session 2 menunggu Session 1. Kondisi tersebut membentuk circular wait dan menyebabkan deadlock.

Ketika SQL Server mendeteksi deadlock, SQL Server akan memilih salah satu transaksi sebagai deadlock victim dan membatalkan transaksi tersebut. Dengan dibatalkannya salah satu transaksi, resource yang ditahannya dapat dilepaskan sehingga transaksi lainnya dapat melanjutkan prosesnya.

N. Penyebab Deadlock

Deadlock dapat terjadi karena beberapa transaksi mengakses resource yang sama dengan urutan yang berbeda. Misalnya, transaksi pertama mengakses PRD001, kemudian PRD002, sedangkan transaksi kedua mengakses PRD002, kemudian PRD001. Selain itu, transaksi yang berjalan terlalu lama dan menahan resource dalam waktu lama juga dapat meningkatkan kemungkinan terjadinya deadlock. Banyaknya transaksi yang berjalan secara bersamaan juga dapat meningkatkan kompleksitas dalam pengelolaan sumber daya.

Oleh karena itu, desain transaksi perlu mempertimbangkan durasi transaksi, jumlah resource yang digunakan, serta urutan akses terhadap resource.

O. Pencegahan Deadlock

Deadlock tidak selalu dapat dihilangkan sepenuhnya, tetapi kemungkinan terjadinya dapat dikurangi. Salah satu cara yang dapat dilakukan adalah membuat transaksi sesingkat mungkin agar sumber daya tidak ditahan lebih lama daripada yang diperlukan. Selain itu, transaksi sebaiknya menggunakan urutan akses terhadap resource yang konsisten. Misalnya, apabila satu transaksi mengakses PRD001 kemudian PRD002, transaksi lain juga sebaiknya menggunakan urutan yang sama.

Penggunaan transaksi yang hanya mencakup operasi yang benar-benar diperlukan juga dapat membantu mengurangi kemungkinan terjadinya deadlock. Selain itu, aplikasi perlu memiliki mekanisme penanganan error sehingga apabila terjadi deadlock, transaksi dapat dibatalkan dan proses dapat ditangani dengan benar.

P. Hubungan Transaction, Locking, Blocking, Isolation Level, dan Deadlock

Transaction Management, locking, blocking, isolation level, dan deadlock merupakan konsep yang saling berkaitan. Transaction digunakan untuk mengelompokkan operasi database menjadi satu kesatuan. Ketika beberapa transaksi berjalan secara bersamaan, SQL Server menggunakan mekanisme locking untuk mengatur akses ke sumber daya. Locking dapat menyebabkan blocking apabila suatu transaksi membutuhkan resource yang sedang digunakan oleh transaksi lain. Isolation level menentukan tingkat isolasi transaksi dan memengaruhi bagaimana transaksi berinteraksi satu sama lain. Dalam kondisi tertentu, transaksi yang saling menunggu sumber daya dapat mengakibatkan deadlock.

Dengan demikian, mahasiswa perlu memahami konsep-konsep tersebut sebagai satu rangkaian. Transaction berhubungan dengan pengelolaan proses pada database. ACID menjelaskan karakteristik transaksi yang dapat diandalkan; locking mengatur akses terhadap resource; blocking menunjukkan kondisi ketika transaksi harus menunggu; isolation level mengatur tingkat isolasi antartransaksi; sedangkan deadlock merupakan kondisi saling menunggu yang harus ditangani oleh sistem.

Q. Penerapan pada Database

Dalam database seperti DB_Petshop, Transaction Management dapat digunakan dalam proses penjualan. Ketika pelanggan melakukan pembelian, sistem dapat menyimpan data ke tabel Penjualan, menyimpan barang yang dibeli ke tabel DetailPenjualan, serta mengurangi

stok pada tabel Barang. Ketiga operasi tersebut dapat ditempatkan dalam satu transaksi. Jika seluruh proses berhasil, transaksi diakhiri dengan perintah COMMIT. Jika salah satu proses gagal, transaksi dapat dibatalkan dengan ROLLBACK.

Dengan penerapan tersebut, Transaction Management membantu menjaga agar database tidak menyimpan transaksi yang hanya berhasil sebagian. Ketika banyak pelanggan melakukan transaksi secara bersamaan, konsep locking dan isolation level menjadi penting untuk mengatur akses ke data. Apabila transaksi saling menunggu resource tertentu, dapat terjadi blocking atau bahkan deadlock.